

ALE 課程 7：Outbound 客製化開發

Lecture

ale06 定義了 IDoc 的形狀，這題要讓資料真的流出去。Outbound 客製化有兩個獨立的問題，ale04 / ale05 已經分別鋪過梗：

1. 什麼時候送？（觸發時機）——Change Pointer / Output Determination / Enhancement 三選一（見 ale05 總表）
2. 怎麼組 IDoc？（組裝與送出）——這題的主軸

組 IDoc 與送出的標準 API：**MASTER_IDOC_DISTRIBUTE**

不管觸發時機是哪一種，最後組好資料、送出去，幾乎都收斂到同一支標準 Function Module

MASTER_IDOC_DISTRIBUTE（2026-09-21 實測介面：**IMPORTING** master_idoc_control LIKE edidc；**TABLES** communication_idoc_control LIKE edidc、master_idoc_data LIKE edidd；例外 **error_in_idoc_control** / **error_writing_idoc_status** / **error_in_idoc_data** / **sending_logical_system_unknown**）。你只需要準備兩樣東西：

- **Control Record (EDIDC)**：最少填 **mestyp** (Message Type) 與 **idocctp** (Basic Type)。不用填接收方——**MASTER_IDOC_DISTRIBUTE** 會依 **Distribution Model (BD64)** 自己決定「這個訊息類型要送給誰」，並針對每個接收方各產生一筆 IDoc（回傳在 **communication_idoc_control** 表裡）。這代表：沒把訊息類型加進 **BD64**，程式不會報錯、只是靜靜地一筆 IDoc 也不產生（**communication_idoc_control** 是空的），這是最常見的「明明呼叫成功卻沒有 IDoc」原因。
- **Data Records (EDIDD)**：一列一個 Segment，最少填 **segnam** (Segment 名稱) 與 **sdata** (整條純文字資料)。做法是先宣告 Segment 對應的 DDIC Structure (ale06 建的 **Z1ALE06H**)、逐欄位賦值，再整個 **MOVE** 進 **sdata**——Segment 是扁平字元結構，這個賦值合法（也是 ale01 / ale06 講過「Segment 全是 **abap.char**」的原因）。父子順序照 Basic Type 定義的階層排列：表頭 Segment 在前、明細 Segment 跟在後面。

COMMIT WORK 的責任歸屬——回顧 en06 的教訓

MASTER_IDOC_DISTRIBUTE 只是把 IDoc 寫進資料庫並登記 tRFC 待送事件，要 **COMMIT WORK** 之後才會真的送出。但 en06 學過：被別人呼叫的程式碼 (BApI、Function Module、Class Method) 不能自己 **COMMIT WORK**。所以本題的分工是：

- **Class 方法** 只負責「讀資料→組 IDoc→呼叫 **MASTER_IDOC_DISTRIBUTE**」，不 **commit**
- 最上層的 **Report** 拿到成功結果之後才 **COMMIT WORK**

這個分工讓同一個 Class 之後可以被不同的觸發時機重用 (ale09 就是從 BApI 掛勾間接呼叫同一套邏輯，那時 commit 由框架負責)。

標準訊息類型的客製化：不重寫，只「插」——Customer Exit

Extension (ale06 Part A) 只是定義了新 Segment 的位置，資料要靠標準的組 IDoc 程式在適當時機呼叫你的 **Exit** 才會被填進去。以 **MATMAS** 為例 (2026-09-21 實測讀 **MASTERIDOC_CREATE_MATMAS** 原始碼確認)：

- 標準程式每組完一個 Segment (如 E1MARAM) 就呼叫一次 CALL CUSTOMER-FUNCTION '002' , 對應 SMOD 增強 MGV00001 的 Function Exit EXIT_SAPLMV01_002 (Include 為 ZXMGVU03)
- Exit 參數 : segment_name (剛組完哪個 Segment) 、 f_mara 等來源表資料 、 idoc_data (整張 IDoc 資料表 , 你可以直接 APPEND 自己的 Segment) 、 回傳 idoc_cimtype (你要告訴系統這筆 IDoc 用了哪個 Extension)
- ⚠ 陷阱 : 這個 Exit 每個 Segment 都會被呼叫一次 , 而 idoc_cimtype 是 EXPORTING 參數——後面幾次呼叫如果不再設定 , 可能把前面設好的值洗掉。保險做法 : 每次被呼叫都設定 idoc_cimtype , 但只有 segment_name = 'E1MARAM' 那一次才 APPEND 你的 Segment 。
- 這個 Exit 屬於 CMOD 專案的範圍 : 建專案→指派 MGV00001→雙擊 EXIT_SAPLMV01_002 生成 Include ZXMGVU03→寫程式→**Activate Project** (回顧 en02 : 光是原始碼啟用不等於生效) 。 ⚠ 系統實測目前 MGV00001 沒有任何 CMOD 專案指派 , 也沒有 ZXMGVU03 , 是乾淨的。

學習目標

- 能講出 MASTER_IDOC_DISTRIBUTE 需要準備什麼、接收方怎麼決定 , 以及「呼叫成功但沒有 IDoc」的排查方向
- 能獨立完成 : Class 方法把採購訂單 (EKKO / EKPO) 組成 ZALE06 IDoc 並送出
- 能遵守 COMMIT WORK 的責任歸屬 : Class 不 commit、最上層 Report 才 commit
- 能用 Customer Exit 替標準訊息類型 (MATMAS) 的 Extension Segment 填值 , 並知道 idoc_cimtype 的陷阱
- 再次練習 Partner Profile Outbound 設定 (這次是全新的自訂訊息類型 ZALE06)

事前準備

- ale02 的自我迴圈管線 (SM59 / WE21 Port / WE20 Partner Profile / BD64 Model ZALE02_MODEL) 已完成
- ale06 的 Segment / Basic Type / Message Type (Z1ALE06H、Z1ALE06I、ZALE06_BT01、ZALE06 , 以及 Part A 的 Z1ALE06 + ZALE06_MATEXT) 已建立、釋出、WE82 已指派 , 且 Field name 與 Data element 同名
- 準備一張既有的採購訂單號碼 (SE16N 查 EKKO 找一張有明細、未刪除的即可) , 只是讀取 , 不會異動它

題目需求

Part A : Partner Profile 與 Distribution Model (GUI · Outbound 再練習)

1. WE20 : 在 Partner S4HCLNT130 (Partner Type LS) 的 **Outbound Parameters** 新增一列——**Message Type ZALE06**、**Receiver Port** 沿用 ale02 的 Port、**Output Mode** Transfer IDoc Immediately、**Basic type ZALE06_BT01**
2. BD64 : 在 ZALE02_MODEL 底下 **Add Message Type**——Sender / Receiver 都是 S4HCLNT130、**Message Type ZALE06**
3. WE20 : 回到原本 MATMAS 的 Outbound Parameters , **Extension** 欄位填 ZALE06_MATEXT (Part C 才會用到 , 先設好)

提醒 Inbound/Outbound 判斷 : 這題只設 Outbound。Inbound 那一半 (接收 ZALE06 要用什麼 Process Code) 是 ale08 的功課——所以本題送出去之後 , 自我迴圈的 Inbound 端因為還沒設定 , IDoc 預期會停在 **Inbound 錯誤狀態** (如 56 / 64) , 這是預期結果 , 不是設定錯誤。

Part B : 自訂 Outbound 程式 (ABAP)

建立 Class **ZCL_ALE07_PO_OUT** 與 Report **ZR_ALE07_SEND** (套件 **\$TMP**) , 規格如下 :

Class **ZCL_ALE07_PO_OUT**

方法	簽章重點	行為
read_po	IMPORTING iv_ebeln TYPE ebeln , RETURNING 一個含「表頭 (ebeln / bukrs / lifnr) + 明細表 (ebelp / matnr) 」的結構	讀 EKKO / EKPO (明細排除已刪除、依 ebelp 排序) ; 找不到採購訂單時回傳表頭為空的結構
map_po	輸入上述結構 , RETURNING EDIDD 標準表	純邏輯、不碰資料庫 : 產生 1 列 Z1ALE06H + 每個明細各 1 列 Z1ALE06I ; 欄位對應 ebeln → ebeln 、 bukrs → bukrs 、 lifnr → elifn 、 ebelp → ebelp 、 matnr → matnr18
send	IMPORTING iv_ebeln , RETURNING 結果結構 (成功 旗標 / IDoc 號碼 / 訊息文字)	組 Control Record (mestyp = ZALE06 、 idocctp = ZALE06_BT01) → 呼叫 MASTER_IDOC_DISTRIBUTE ; 不可 COMMIT WORK ; PO 不存在、無明細、ALE 例外、沒有接收方 (communication_idoc_control 為空) 四種情況都要回傳明確的失敗訊息

Report ZR_ALE07_SEND : 選取畫面 **p_ebeln** (必填) + **p_test** (勾選 = 只顯示組出來的 Segment 內容、不送出、預設勾選) ; 未勾選時呼叫 **send** , 成功才 **COMMIT WORK** , 並 **WRITE** 出 IDoc 號碼。

ABAP Unit : 替 **map_po** 寫測試類別——餵入手工組出的「1 表頭 + 2 明細」, 驗證產生 3 列、**segnam** 順序 (表頭在前)、**sdata** 開頭正確帶出採購訂單號碼與項次 (這個方法不碰資料庫, 測試不需要真實採購訂單)。

Part C : 替 **MATMAS** 的 Extension 填值 (Customer Exit , GUI + ABAP)

1. **MM02** 替 **ale02** / **ale04** 用過的物料 **000000000000000021** 填入 **Season Year** (**MARA-SAISJ** , 例如 **2026**) —— ⚠ **真實資料異動** , 先記下原值, 驗證完成後還原
2. 建立 Class **ZCL_ALE07_MATMAS_EXT** , 提供類別方法 **fill_extension** : 輸入 **segment_name** 與 **MARA** 資料, **CHANGING** IDoc 資料表與 **cimtype** ; 每次呼叫都設定 **cimtype** = **ZALE06_MATEXT** , 僅當 **segment_name** = **E1MARAM** 時 **APPEND** 一列 **Z1ALE06** (**saisj** / **datab** 取自 **MARA**)
3. **CMOD** : 建專案 **ZALE07** → **Enhancement Assignment** 指派 **MGV00001** → **Components** 雙擊 **EXIT_SAPLMV01_002** 生成 Include **ZXMGVU03** (生成後 Claude 才能用 ADT 寫入內容, 寫法就是呼叫 **fill_extension**) → **Activate Project**
4. 觸發 **MATMAS** 分送 (**BD10** 直接送指定物料, 或依 **ale04** 的 Change Pointer 流程) , 用 **WE02** 檢視新 IDoc : Control Record 的 **Extension** 欄位應為 **ZALE06_MATEXT** , **E1MARAM** 底下應出現 **Z1ALE06** Segment 並帶著 Season Year

⚠ **Segment 順序陷阱 (回頭驗證項目)** : IDoc 資料表裡 Segment 的出現順序必須與 **WE30** 結構定義的順序一致, 否則語法檢查會失敗 (IDoc 狀態 **26**)。本題的 Exit 是在 **E1MARAM** 剛組完、它的其他子 Segment (如 **E1MAKTM**) 還沒組之前被呼叫, 所以 **APPEND** 的 **Z1ALE06** 會成為 **E1MARAM** 的第一個子 Segment。請用 **WE60** 確認 **ale06** 的 Extension 裡 **Z1ALE06** 排在 **E1MARAM** 子節點的哪個位置——如果排在其他兄弟 Segment 之後, Exit 要改成「等到最後一個兄弟 Segment 那次呼叫」才 **APPEND**。這個實

作細節未實測，請把 WE02 檢視到的實際結果回報。(ale10 的 INVOIC 則相反：Exit 在整張 IDoc 組完後才呼叫，所以要用 INSERT ... INDEX 插到指定位置。)

參考答案 (驗證方式)

參考答案程式碼已快照在本目錄：`zcl_ale07_po_out.clas.abap`、`zcl_ale07_po_out.clas.testclasses.abap`、`zr_ale07_send.prog.abap`、`zcl_ale07_matmas_ext.clas.abap`、`zxmguv03.prog.abap`。

⚠ 狀態：草稿，尚未經 SAP 語法檢查與執行驗證——這些程式碼引用 ale06 才會建立的 Segment 結構 (Z1ALE06H / Z1ALE06I / Z1ALE06)，要等 ale06 的 GUI 步驟完成後才能推送到 SAP 編譯。待驗證清單見 README「待驗證項目盤點」。

驗證要點 (完成後回報，我會用字典表核對)：

- `SELECT DOCNUM, MESTYP, IDOCTP, CIMTYP, STATUS FROM EDIDC WHERE MESTYP IN ('ZALE06','MATMAS') ORDER BY DOCNUM DESCENDING`
- `SELECT SEGNUM, SEGNAM, PSGNUM, SDATA FROM EDID4 WHERE DOCNUM = '<IDoc號>'`——ZALE06 應為「1 個 Z1ALE06H + N 個 Z1ALE06I (PSGNUM 指向表頭)」；MATMAS 應在 E1MARAM 之後出現 Z1ALE06
- `SELECT MESTYP, RCVPRN, RCVPOR, IDOCTYP, CIMTYP FROM EDP13 WHERE MESTYP IN ('ZALE06','MATMAS')`——核對 Outbound Partner Profile

思考題

1. `send` 為什麼不直接在方法裡 `COMMIT WORK`？如果將來要在 BAdI 掛勾裡重用這個方法，這個設計有什麼好處？(提示：回顧 en06 的 MESSAGE_TYPE_X Dump)
2. `MASTER_IDOC_DISTRIBUTE` 在沒有接收方時「不報錯、也不產生 IDoc」。從監控角度，這種「靜默失敗」比明確報錯更難處理——你會在 `send` 裡怎麼設計，讓呼叫者一定知道「沒有送出任何東西」？
3. Customer Exit 每個 Segment 呼叫一次、`idoc_cimtype` 可能被後續呼叫洗掉——這個設計如果由你來重新設計，你會怎麼改，讓 Exit 開發者更不容易踩到陷阱？(提示：想想「輸出參數的預設值該是什麼」)

答案

Part A 為 GUI 設定；Part B / C 的參考答案程式碼見上方快照檔。Part C 的 CMOD 專案、Enhancement Assignment 與 Activate Project 為 GUI-only (無 ADT API，見 `.claude/rules/sap-adt-mcp.md` 第 20 / 21 節)，請完成後回報。